Práctica de PROCESADORES DE LENGUAJES

Doble Grado Ingeniería Informática-Matemáticas
Facultad de Informática
Profesor: Albert Rubio

febrero de 2025

1. Introducción

Las prácticas se harán en grupos de un máximo de tres estudiantes y solo debe hacer la entrega uno de los miembros del grupo. Los miembros del grupo deberán indicarse en el documento de especificación entregado y serán los mismos durante todo el desarrollo de la práctica (salvo excepciones muy justificadas).

Todas las prácticas tienen en común la definición de un mini-lenguaje de programación, y el desarrollo de un compilador para el mismo que cubra las fases vistas en clase. Las prácticas se harán en grupos de hasta tres estudiantes. Salvo excepciones acordadas con el profesor, el lenguaje de desarrollo será Java, y el lenguaje máquina generado por el compilador será WebAssembly (en su formato textual). Las fases de análisis léxico y sintáctico se harán utilizando las herramientas JFlex y CUP respectivamente. El resultado de las mismas será un árbol de sintaxis abstracta. Estas fases, y las de análisis de la semántica estática, informarán de los posibles errores, y en caso de haberlos no se generará código máquina. Si no hay errores, el compilador generará un fichero de texto con el código WebAssembly ejecutable desde Javascript (tal como se indica en la documentación de las sesiones de laboratorio).

A continuación se detallan los requisitos mínimos de cada faceta del lenguaje y los opcionales. Dadas las restricciones temporales, se aconseja ser prudente a la hora de elegir alguno(s) de estos últimos.

2. Requisitos mínimos y opcionales del lenguaje diseñado

2.1. Nombre

Es obligatorio que el lenguaje tenga un nombre (sin separadores).

2.2. Sintaxis del lenguaje

No busquéis una sintaxis muy parecida a C++ o Java, ya que complica la gramática. Por ejemplo, en las declaraciones es mejor que el tipo vaya detrás de la variable con algún

símbolo que separe las dos cosas. También se puede añadir alguna palabra clave antes de cada declaración. Añadir palabras clave y símbolos facilita la definición de la gramática.

2.3. Identificadores y ámbitos de definición

Mínimos:

- Declaración de variables simples y de arrays de cualquier tipo, incluidos otros arrays (o bien, permitir arrays de varias dimensiones).
- Bloques anidados, que requerirá trabajar con tabla de símbolos para bloques anidados.
- Funciones con paso de parámetros por valor o por referencia.

Opcionales: Punteros, registros, clases (sin ningún tipo de herencia). Módulos, cláusulas de importación. Definición de funciones anidadas.
Incompatibles: No incluir registros y clases (o una cosa o la otra o ninguna). Si se incluyen funciones anidadas, no hacer ni módulos ni clases.
Grupos de 3: Es muy recomendable o bien incluir punteros y registros, o bien clases.

2.4. Tipos

Mínimos:

- Declaración explícita del tipo de las variables.
- Tipos básicos predefinidos enteros y booleanos.
- Operadores infijos, con distintas prioridades y asociatividades para estos tipos.
- Tipo array de tamaño fijo.
- Comprobación de tipos.

Opcionales: Definición de tipos de usuario. Tipo Real. Tipo Enumerado.
Incompatibles: Solo incluir máximo una de las tres cosas.

2.5. Conjunto de instrucciones del lenguaje

Mínimos:

- Instrucción de asignación incluyendo asignaciones a una posición de un arrays.
- Condicional con una y dos ramas, y algún tipo de bucle.
- Expresiones formadas por constantes, identificadores con y sin subíndices (para acceso a arrays), operadores infijos y llamadas a función (y métodos de clase si se incluyen clases).

Opcionales:

- Si habéis incluido punteros o registros (o clases) entonces hay que añadir expresiones con punteros y nombres cualificados (notación "."). Instrucciones de reserva de memoria dinámica (en presencia de punteros).
- Instrucción case o similar con salto a cada rama en tiempo constante.
- Declaraciones de variables con inicialización para tipos básicos.
- Declaraciones de variables con inicialización para estructurados (arrays). Implica tener constantes de tipo array.

Grupos de 3: Es muy recomendable incluir todo lo relacionado con punteros y registros.

2.6. Gestión de errores

Mínimos: Indicación del tipo de error, fila y columna. Parar la compilación. Recuperación de errores (tratar de proseguir la compilación tras un error, a fin de detectar más errores).

3. Entregas

Las entregas se harán a través del Campus Virtual por uno de los miembros del grupo. Las tareas se cerrarán a las 23:59 del día especificado como límite para cada una.

1. Especificación en papel de la sintaxis del lenguaje y de ejemplos típicos de programas escritos en dicho lenguaje. Esta entrega, si se hace dentro de plazo, se revisará en clase de laboratorio. Fecha límite: 23 de febrero.
2. Ficheros de especificación del léxico y de la sintaxis, con atributos para la construcción del árbol de sintaxis abstracta, procesados con éxito por JFlex y CUP. Fecha límite: 6 de abril.
3. Código Java completo para el análisis sintáctico, todos los análisis semánticos y la generación de código WebAssembly y ejemplos de prueba. Fecha límite: 18 de mayo. Pre-entrega 11 de mayo. El día 11 de mayo se debe hacer una pre-entrega de una versión incompleta de la práctica pero todo lo funcional que se pueda para código sencillo. Esta versión servirá para detectar problemas importantes que impidan la evaluación de la práctica. En tal caso, el grupo recibirá un mensaje indicando los problemas.